GEO1016 Assignment 1: Camera Calibration

1 Introduction

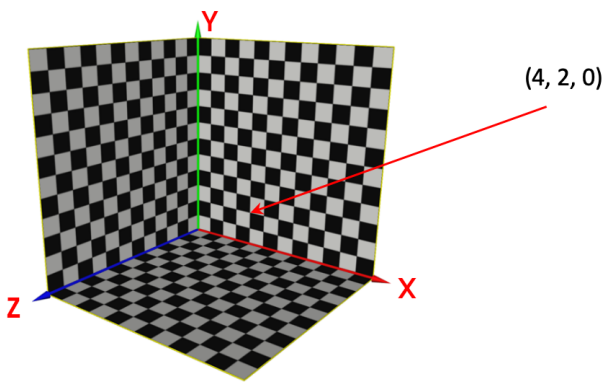
In this assignment, you will gain hands-on experience in camera calibration. Specifically, you will practice:

- Basic linear algebra operations (e.g., matrix-vector product and SVD).
- Collecting a set of 3D-2D corresponding points using a (virtual) apparatus.
- Extracting intrinsic and extrinsic camera parameters.
- Using open-source libraries to visualize and validate your calibration results.

2 Tasks

2.1 Collect at least 6 pairs of 3D-2D corresponding points (10%)

Compile and run the viewer, you will see a virtual calibration rig like



In this rig, the red, green, and blue arrows denote the X, Y, and Z axes, respectively. With the grid texture, you can easily figure out the 3D coordinates of any cell corners on the grid. You can use your mouse to manipulate the scene (i.e., the calibration rig). Take a picture of it by pressing the `s` key. Then you will see a dialog asking for a file name. You can provide any name, e.g., `image_01.png`, and save it to your disk. With the image, pick at least 6 points on the virtual scene and determine their corresponding image coordinates. Store the correspondences in a `.txt` file, with each line containing five coordinates separated by spaces, i.e.,

```
x y z u v
```

where `x, y, z` are the 3D coordinates, and `u, v` are the corresponding 2D image coordinates. See the provided data files in `/resources/data`.

- **Data**. Use both the provided data and your own data for testing your camera calibration algorithm. We assume every team collects a different set of data for the experiment.
- **Image coordinates**. The image below shows the image coordinate system. Image coordinates are denoted in pixels, with the origin point (0, 0) corresponding to the top-left corner of the image. The X-axis starts at the left edge of the image and extends towards the right edge. The Y-axis starts at the top and extends downwards. All image pixels have non-negative coordinates. Many image viewers and editing tools can help you easily identify the coordinates of the pixel under the cursor, e.g., GIMP and ImageJ (both are open source and cross-platform).

2.2 Implement the camera calibration algorithm (50%)

Implement the `calibration` function in `calibration_method.cpp`. In the provided code framework, this function is defined without the algorithm implementation. Comments and guidance are left in the function. Example code is also provided within the function for you to get familiar with the necessary data structures and algorithms. To implement the camera calibration algorithm, the following subtasks must be completed:

- Construct the P matrix. (10%)
- Solve for M, the whole projection matrix (i.e., $M = K[R|\mathbf{t}]$). (10%)
- Extract intrinsic parameters from M. (10%)
- Extract extrinsic parameters from M. (10%)
- Explicitly determine the sign of ρ. (10%)

2.3 Evaluation (10%)

- Evaluate your calibration results qualitatively and quantitatively. (5%)
- Invite students from another group to review your code. Incorporate the received feedback in your implementation and reflect on the changes in the report. (5%)

Note: You're encouraged to discuss and share experiences with other groups, but copying codes from others will be penalized.

3 Submission

Your submission should include:

1) **Report (30%)** (Max 3 pages excluding figures and tables).

   You're expected to deliver a clear, precise, and well-structured scientific report. Make every sentence, equation, and notation precise and clear to avoid misinterpretation. Meanwhile, the report should be as concise as possible, but it should provide key information to reimplement the method to reproduce your results, and it should at least include:

   - Description of the methodology with mathematical language (e.g., equations). (5%)
   - Calibration results on both provided and collected data. (5%)
   - Screenshots of calibration results shown next to input images. You should use the screenshot function provided by the viewer for this. **Note**: the resolution of the screenshots may be affected by the DPI scaling factor of your display (e.g., a MacBook with a retina display will generate images with doubled resolution, which may result in different intrinsic parameters). Therefore, it is recommended to take the input and resulting images on the same machine or machines with the same DPI scaling factor. (3%)
   - Explanation of intermediate result verification (if necessary). (2%)
   - Discussion on determining the sign of ρ, given the world coordinate system used in this assignment. (5%)
   - Discussion on the accuracy of your results, e.g., Can you measure the errors? How to improve accuracy? (5%)
   - Reflection on how the feedback received from others helped to improve your implementation. (5%)
   - Workload distribution (i.e., who did what) using a few bullet points for each member.

   To promote formal scientific writing, the following rules apply for assessment:

   - Any misunderstanding or misconception of main concepts: 10% deduction.
   - Multiple unclear or ambiguous descriptions: 10% deduction.
   - Multiple typos, grammar issues, format issues (e.g., a figure/table without a caption or multiple figures/tables with the same caption, unindexed or unreferenced figures/tables), consistency issues (e.g., upper case and lower case used interchangeably, regular font and italic font used interchangeably), misuse of symbols in notations: 10% deduction.
   - If the report exceeds the length limit by more than half a page: 10% deduction.

2) **Data**. This includes:

   - Your collected 3D-2D point pairs (in `.txt` format).
   - Your collected image (taken using the screenshot function of the viewer).

3) **Source code**. Only the following file is required:

   - `A1_Calibration_Code/Calibration/calibration_method.cpp`

Note: your source code should compile and reproduce your results without modification. Additionally, please try to test your code on different operating systems (if available), as it may be evaluated on a system different from the one used for your development.

How to submit? Compress all required files into a single archive named in the following format:

GEO1016_A1_Group_XX.zip

where "XX" is the 2-digit ID of your group ID (e.g., 03). If you submit an updated version, append a version number, e.g., "_v2", to the file name.

Appendices

Appendix 1: About the Code Framework

The code framework is based on **Easy3D**[1]. The viewer provides visualization of 3D scenes and cameras. It also enables intuitive validation of calibration results. The code framework uses a stripped earlier version of Easy3D, which is not compatible with its latest version.

Appendix 2: Image Comparison Tool

Tools such as ImageDiff[2] can help visualize differences between images. Pre-built executables are available for Mac, Windows, and Linux.

[1] `https://github.com/LiangliangNan/Easy3D`
[2] `https://github.com/LiangliangNan/ImageDiff`